
Homework 4 Paper Questions

ResNets and Transformers

Deliverables. Submit a single PDF of your write-up to Gradescope *HW4 Paper Questions*.

Note: we highly discourage very long answers, most or all of your free response answers should be 1-3 sentences.

Honor Code

Write and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature: Jingliang Tu

1 ResNets

Problem

Q1. (1 Pt.) In less than 5 words, describe the kind of tasks was ResNet designed for.

Answer: Image classification and recognition.

Q2. (1 Pt.) What problem did the authors of the ResNet paper aim to solve?

Answer: They aimed to solve the “degradation problem,” where adding more layers to a deep neural network causes training accuracy to saturate and then degrade. Unlike overfitting, this degradation leads to higher training error, indicating that deep networks are simply harder to optimize.

Current works

Q3. (2 Pts.) How might you construct a deeper counterpart of a neural network that produces the exact same outputs as a shallower network? Why doesn’t this work in practice, i.e. why does degradation occur in deeper neural networks?

Answer: You could theoretically construct a deeper network by taking a shallower trained network and adding identity mapping layers on top. In practice, optimization difficulties prevent current solvers from successfully learning these identity mappings, leading to vanishing/exploding gradients and higher training error (degradation).

Proposed Solution

Q4. (1 Pt.) The ResNet authors proposed building layers that can learn the residual function $F(x) + x$ instead of directly learning $H(x)$. Why might we want to learn the function $F(x) + x$ if it is equivalent to $H(x)$? How does learning $F(x)$ help solve the degradation problem?

Answer: Learning the residual mapping $F(x) = H(x) - x$ makes it easier for the solver to approximate an identity mapping, as it just needs to drive the layer’s weights to zero. This solves degradation by ensuring that adding deeper layers does not perform worse than shallower counterparts, because the network can easily bypass unnecessary layers via the identity shortcut.

Method Details

Q5. (2 Pts.) What are two workarounds if your original input x and the output of your sub-layer $F(x)$ don’t have equal dimensions?

Answer: The two workarounds are: performing zero-padding to increase dimensions (which introduces no extra parameters), or using a linear projection (e.g., 1×1 convolutions) to match the dimensions before addition.

Q6. (2 Pts.) Explain the 3 options for how residual/shortcut connections might be implemented and compare their parameter counts. Based on the parameter count comparisons, briefly

discuss how the 3 different residual connection implementations might affect the model's memory requirements or training time.

Answer: Option A uses zero-padding for increasing dimensions and identity shortcuts elsewhere (least parameters). Option B uses projection shortcuts only when dimensions increase (moderate parameters). Option C uses projection shortcuts for all connections (most parameters). Option C has the highest memory footprint and longest training time due to extra parameters, while A is the most efficient; B offers a solid balance of computational efficiency and accuracy.

Key Insight & Contributions

Q7. (1 Pt.) What benefits did the authors notice with residual networks? What empirical proof did the authors provide to show that ResNet architectures didn't suffer from the degradation problem?

Answer: The authors found that residual networks are easier to optimize and can successfully gain accuracy from substantially increased depth. As empirical proof, their 152-layer ResNet achieved lower training and test errors than shallower counterparts, successfully reversing the degradation trend seen in plain networks.

2 Transformers - Attention Is All You Need

Problem

Q8. (1 Pt.) In a short sentence, What task is the paper “Attention Is All You Need” trying to solve?

Answer: The paper aims to solve sequence transduction tasks, specifically machine translation.

Metrics

Q9. (2 Pts.) Two important metrics that the authors highlight in the paper are BLEU scores and perplexity. Describe what BLEU score and perplexity measure, and what their limitations are.

Answer: BLEU score measures the n-gram overlap between generated and reference translations, but it is limited because it ignores semantics, grammar, and penalizes valid paraphrasing. Perplexity measures how well a probability distribution predicts a sample (lower is better), but it only evaluates statistical likelihood and does not guarantee the human-perceived fluency or contextual quality of the text.

Current Works

Q10. (1 Pt.) What are some challenges that existing architectures face? Describe the shortcomings of both recurrent neural networks (RNNs) and sequential convolutional neural networks.

Answer: RNNs are strictly sequential, which prevents parallelization within training examples and causes memory bottlenecks for long sequences. Sequential CNNs require computational effort that grows linearly or logarithmically with distance, making it very difficult for them to learn long-range dependencies.

Proposed solution

Q11. (1 Pt.) In a short sentence, describe what “attention” is in your own words.

Answer: Attention is a mechanism that allows a model to selectively focus on the most relevant parts of the input context, dynamically weighing different elements based on their importance to the current task.

Q12. (1 Pt.) What are the jobs of the encoder and decoder?

Answer: The encoder maps the input sequence into a sequence of continuous contextual representations. The decoder then uses these representations along with previously generated tokens to auto-regressively generate the final output sequence.

2.1 Method Details

Q13. (1 Pt.) What is the purpose of masking during decoder self-attention?

Answer: Masking prevents the decoder from attending to future positions in the sequence during training. This preserves the auto-regressive property, ensuring that predictions for position i only depend on known outputs at positions before i .

Q14. (2 Pts.) Why does the Transformer paper scale QK^T by $1/\sqrt{d_k}$?

Answer: If elements of Q and K are independent random variables with variance 1, their dot product will have a variance of d_k , leading to extremely large magnitudes. Scaling by $1/\sqrt{d_k}$ restores the variance to 1, preventing the dot products from pushing the softmax function into regions with vanishing gradients.

Q15. (1 Pt.) What are 2 benefits of multi-headed attention?

Answer: First, it allows the model to jointly attend to information from different representation subspaces simultaneously. Second, it allows the model to capture different types of relationships across different positions, which single-head attention struggles to do due to weight averaging.

Q16. (2 Pts.) Where do the query, key, and value vectors come from in the following parts of a transformer, and why?

Answer:

- (a) **Encoder self-attention:** Q, K, and V all come from the output of the previous encoder layer, allowing attention over all input positions.
- (b) **Decoder self-attention:** Q, K, and V come from the previous decoder layer, allowing attention over previously generated tokens.
- (c) **Decoder cross-attention:** Q comes from the previous decoder layer, while K and V come from the output of the encoder, allowing the decoder to attend to the entire input sequence.

Q17. (2 Pts.) Explain the differences between what the transformer decoder receives as input and what it outputs during training vs. inference.

Answer: During training, the decoder receives the entire ground-truth sequence shifted right (teacher forcing) and predicts all tokens simultaneously in parallel. During inference, it operates auto-regressively, receiving the sequence generated so far to predict just the single next token, looping this process sequentially.

Q18. (2 Pts.) Why does the standard input to a transformer model (before adding positional encodings) not contain any information about the order of tokens in a sequence? How do positional encodings help the model understand token order?

Answer: The self-attention mechanism performs a permutation-invariant operation, computing pairwise interactions identically and treating the input purely as a “bag of words.” Positional encodings are added to the input embeddings to inject explicit absolute and relative position information, allowing the model to distinguish token order mathematically.

Key Insight / contributions

Q19. (2 Pts.) What advantages does the self-attention mechanism have over recurrent layers and convolutional layers in the context of sequence modeling?

Answer: Self-attention has a lower total computational complexity per layer when sequence length is smaller than the representation dimensionality. It also allows for massive parallelization requiring only $O(1)$ sequential operations, and easily captures long-range dependencies due to its $O(1)$ maximum path length.

Future Work

Q20. (2 Pts.) How might you adapt transformers to process images? Think about how you might create “tokens” from images.

Answer: An image can be split into a grid of fixed-size, non-overlapping patches (e.g., 16×16 pixels). These patches are flattened and linearly projected into 1D continuous vectors to serve as “tokens,” which are then appended with positional embeddings and fed into the standard Transformer architecture.

Q21. (2 Pts.) Suppose you have a ResNet and a Vision Transformer, both trained on ImageNet... Now, you are given a new image of size 1024×512 . How would you process this image for each model? What are the key differences in how ResNets and Vision Transformers handle input image sizes?

Answer: A ResNet is fully convolutional, so it can naturally process the varying dimensions by producing a larger spatial feature map that is simply collapsed by the Global Average Pooling layer. Conversely, a Vision Transformer will generate a vastly larger number of patches (2048), which requires 2D interpolating the pre-trained positional embeddings and scaling the computationally expensive $O(n^2)$ self-attention mechanism to handle the massively increased sequence length.